

Smart Asset Management and Resource Allocation Platform

Design Document

Cultural Council Open Projects 2026, IIT Roorkee

Developer: Rishi Chaudhary

Date: June 2026

Table of Contents

1. Problem Understanding
 2. System Architecture
 3. Database Schema
 4. Entity Relationship Diagram (ERD)
 5. API Overview
 6. Design Decisions
-

1. Problem Understanding

Background

The Cultural Council at IIT Roorkee has a huge inventory of common physical resources that include DSLR Cameras, Studio Lights, Audio Equipment, Costumes, Stage Props, Recording Gears, and Events Resources. These are regularly used by more than one section/event within the year.

Current Challenges

The existing resource management process relies on spreadsheets, manual registers, and informal coordination. This results in:

- & No ability to monitor resource availability in real-time

- & Frequent scheduling conflicts among different sections
- & Accountability not ensured with delayed returns
- & Under-utilized resource because of inefficient discovery process
- & Absence of data for future events planning

Proposed Solution

A full-stack web-based platform that converts the entire cycle into digital form – discovery, reservation, approval, issuance, and return. It has two types of user access: **Admin** (Cultural Council coordinator) and **User** (Section member/Event organizer).

Goals

- & Inventory visibility to everyone in real time
 - & Eliminate ad-hoc approvals by structuring them
 - & Keep updated inventory counts all the time
 - & Enable data analysis for resource allocation
 - & Eliminate paper-based practices by digitalizing them
-

2. System Architecture

Overview

The platform follows a standard **3-tier architecture**:

- & **Presentation Layer:** React.js (Frontend)
- & **Application Layer:** Node.js + Express.js (Backend API)
- & **Data Layer:** MongoDB Atlas (Cloud Database)

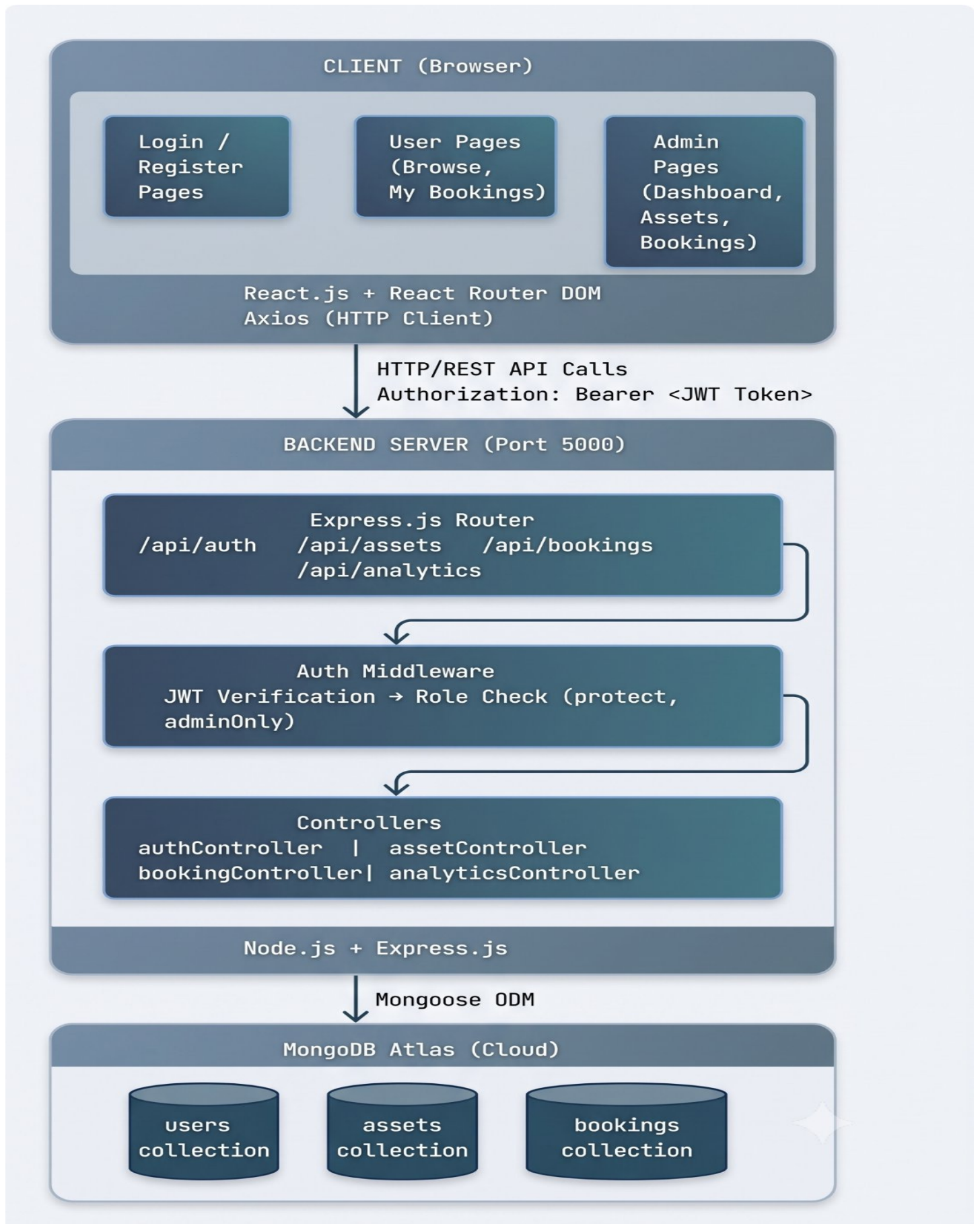
Key Architectural Decisions

Separation of concerns: Frontend and backend are completely separate applications communicating via REST APIs

Stateless authentication: JWT tokens guarantee that session state is not needed by the backend

Cloud database: MongoDB Atlas removes infrastructure concerns and offers high availability

Architecture Diagram



3. Database Schema

3.1 Users Collection

users {		
_id:	ObjectId	
name:	String	
email:	String	
password:	String	
role:	String	(enum: ['user', 'admin'], default: 'user')
createdAt:	Date	
updatedAt:	Date	
}		

3.2 Assets Collection

assets {		
_id:	ObjectId	
name:	String	
category:	String	(enum: Camera, Lighting, Audio, Costume, Stage Props, Recording, Event Infrastructure, Other)
description:	String	
totalQuantity:	Number	
availableQuantity:	Number	
status:	String	(enum: ['Available', 'Unavailable'])
addedBy:	ObjectId	(ref: User)
createdAt:	Date	
updatedAt:	Date	
}		

3.3 Bookings Collection

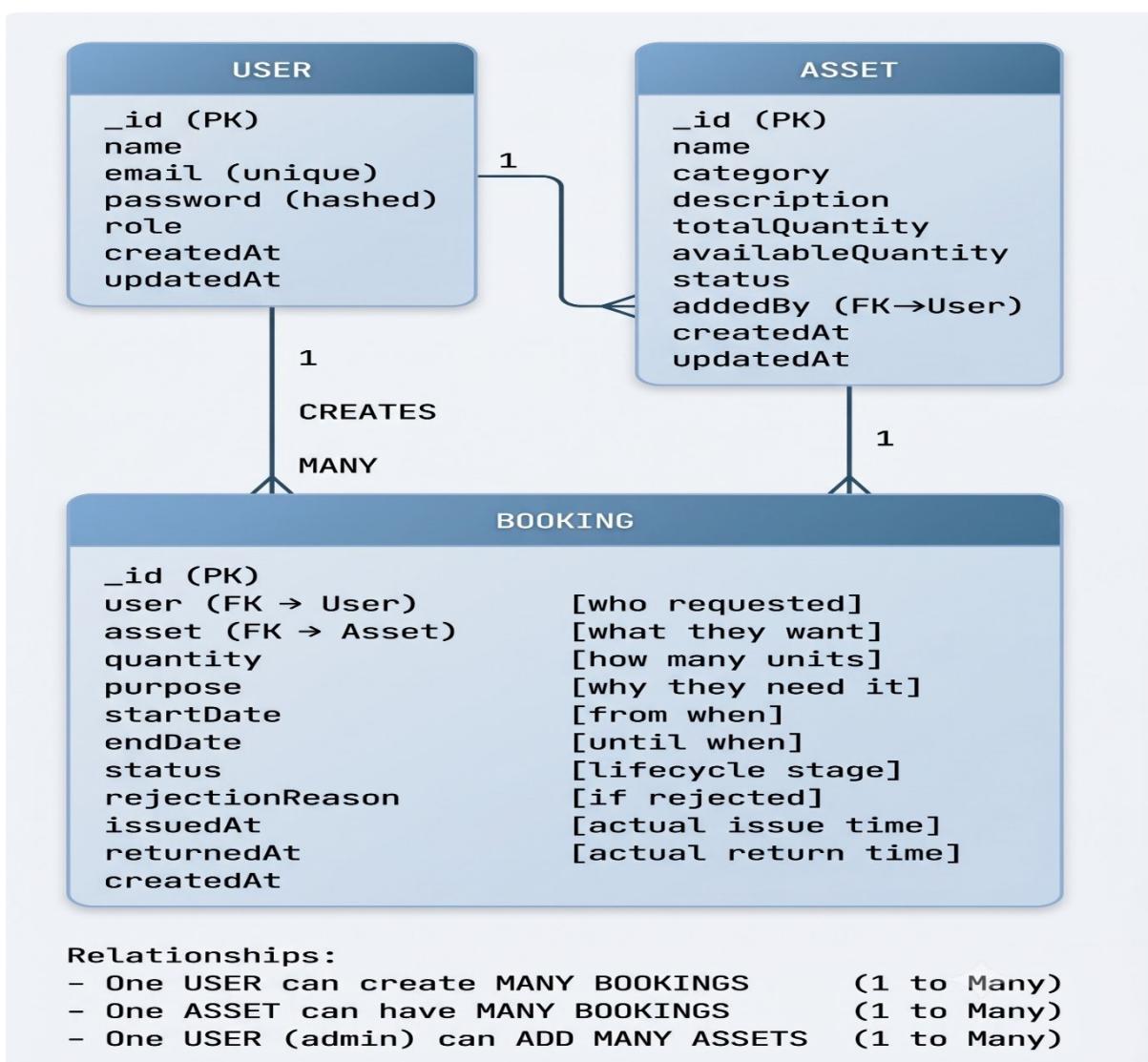
```

bookings {
  _id:                ObjectId
  user:               ObjectId    (ref: User)
  asset:              ObjectId    (ref: Asset)
  quantity:           Number
  purpose:            String
  startDate:          Date
  endDate:            Date
  status:             String      (enum: pending → approved/rejected
                                   approved → issued issued →
                                   returned)

  rejectionReason:    String
  issuedAt:           Date        (set when admin issues asset)
  returnedAt:         Date        (set when asset is returned)
  createdAt:          Date
  updatedAt:          Date
}

```

4. Entity Relationship Diagram (ERD)



5. API Overview

Authentication Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new user account
POST	/api/auth/login	Public	Login and receive JWT token

Asset Endpoints

Method	Endpoint	Access	Description
GET	/api/assets	Protected	Get all assets
GET	/api/assets/:id	Protected	Get single asset by ID
POST	/api/assets	Admin only	Add new asset to inventory
PUT	/api/assets/:id	Admin only	Update asset information
DELETE	/api/assets/:id	Admin only	Remove asset from inventory

Booking Endpoints

Method	Endpoint	Access	Description
POST	/api/bookings	Protected	Submit new booking request
GET	/api/bookings	Admin only	Get all bookings system-wide
GET	/api/bookings/my	Protected	Get current user's bookings
PUT	/api/bookings/:id/approve	Admin only	Approve pending booking
PUT	/api/bookings/:id/reject	Admin only	Reject booking with reason
PUT	/api/bookings/:id/issue	Admin only	Mark asset as physically issued
PUT	/api/bookings/:id/return	Admin only	Mark asset as returned

Analytics Endpoints

Method	Endpoint	Access	Description
--------	----------	--------	-------------

GET	/api/analytics/dashboard	Admin only	Get all dashboard statistics
-----	--------------------------	------------	------------------------------

Authentication Flow

1. User registers → password hashed with bcrypt → stored in DB
 2. User logs in → password compared with bcrypt.compare()
 3. Server generates JWT token (payload: userId + role, expires: 7 days)
 4. Client stores token in localStorage
 5. Every protected request includes: Authorization: Bearer <token>
 6. authMiddleware verifies token → attaches req.user
 7. adminOnly middleware checks req.user.role === 'admin'
-

6. Design Decisions

6.1 Why MongoDB over SQL?

MongoDB was chosen because asset data is flexible and schema changes are frequent during development. The document model naturally represents nested data like booking history. MongoDB Atlas also provides a free managed cloud tier, eliminating infrastructure overhead for this project.

6.2 Why JWT over Session-Based Auth?

JWT tokens are stateless, the server doesn't need to store session data. This makes the backend scalable since any server instance can verify any token using the secret key. Sessions require shared storage across server instances, which adds complexity.

6.3 Inventory Consistency Strategy

The most critical design challenge was maintaining accurate inventory counts. The approach taken:

- & **On approval:** availableQuantity decreases immediately (prevents double-booking)
- & **On return:** availableQuantity increases back
- & **Availability check on approval:** double-checks quantity in case of concurrent requests
- & **Status auto-update:** asset marked Unavailable when availableQuantity hits 0

6.4 Overdue Calculation

Rather than comparing today's date with the originally requested end date, overdue is calculated as:

$\text{actualDueDate} = \text{issuedAt} + (\text{endDate} - \text{startDate})$

This is fairer, if admin issues an asset late, the user still gets their full requested duration.

6.5 Role-Based Access Control

Two roles are implemented — User and Admin. Protected routes use middleware stacking:

```
router.put('/:id/approve', protect, adminOnly, approveBooking)
```

This ensures every admin route passes through two layers of verification before reaching the controller.

6.6 Why React + Vite over Next.js?

For this project, a pure client-side React app is sufficient. Next.js adds server-side rendering complexity that isn't needed here. Vite provides extremely fast hot module replacement during development, improving development speed significantly.

6.7 Scalability Considerations

- & **Database indexing:** Email field on Users is unique-indexed, asset and user fields on Bookings are indexed via Mongoose refs
- & **Modular architecture:** Controllers, routes, models and middleware are fully separated, adding new features requires no changes to existing code
- & **Environment variables:** All secrets and config are in .env, making deployment to any environment straightforward